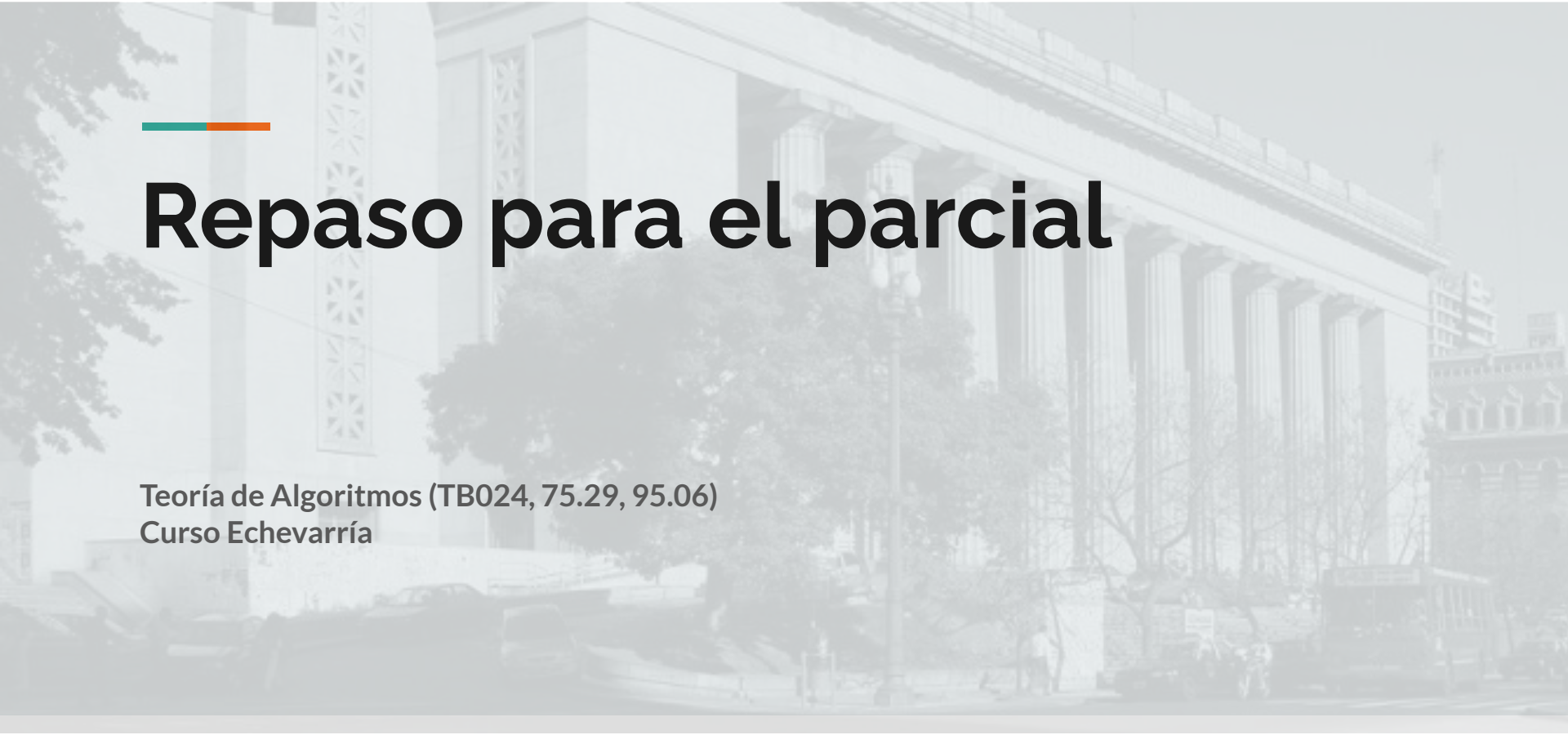


Repaso para el parcial

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

¿Qué es lo que vimos hasta acá?



Repaso

- Problemas
- Algoritmos que los resuelven
- Técnicas para diseñar esos algoritmos
- Conceptos Teóricos

Técnicas



Créditos: Andrea Piacquadio

División y Conquista

1. Dividir un problema mayor en una serie de problemas menores.
2. Resolver cada uno de estos problemas.
3. Agrupar los resultados y obtener la solución.

- Se arma top-down
- Esta división suele ser recursiva
- El Teorema Maestro nos da una cota superior de complejidad temporal

Algoritmos Greedy

1. En cada paso, tomar la acción que mejor resultado parcial tiene, sin importar sus implicancias futuras.
2. Mientras falten pasos, volver a 1

- Muy buenos pasos iniciales, muy malos los últimos
- A FAVOR: Son intuitivos, fáciles de programar y rápidos
- EN CONTRA: En muchos casos no encuentran el óptimo (y pueden quedar muy lejos)

Backtracking

1. Genera un árbol de soluciones parciales.
2. Si una solución parcial ya no puede llegar a ser válida, “poda” esa rama del árbol y deshace el último paso, retrocediendo.

- El problema debe tener soluciones parciales incrementales, y que sean fáciles de computar
- Para optimizar las podas, es fundamental tener buenas cotas
- El resultado es óptimo (puede demorar)

Programación Dinámica

1. Resuelve el primer paso del problema.
2. Para el siguiente paso, define si es mejor darlo o no, teniendo en cuenta el resultado del paso anterior.
3. Se repite hasta terminar.

- Se arma bottom-up
- Subestructura óptima: La solución del problema se arma a partir de las soluciones de los subproblemas
- Subproblemas superpuestos: Cada subproblema incluye al anterior
- Memoization: Se agiliza memorizando las soluciones parciales

Programación lineal

- No elabora un algoritmo, sino un modelo matemático
- Este modelo está formado por ecuaciones e inecuaciones lineales
- Las variables pueden ser continuas, enteras o binarias
- Siempre encuentra el óptimo, pero puede tardar (especialmente con variables enteras)
- Los problemas continuos se resuelven por el método *Simplex*
    - Su complejidad, en el peor caso, es exponencial, pero en la práctica funciona mucho más rápido
- Los problemas enteros agregan *Planos de Corte* o *Branch and Bound* para podar

Reducciones

1. $X \leq_P Y$: Reformular el problema X como un problema Y
2. Resolverlo con los métodos para problemas de tipo Y
3. Convertir la solución de Y en una solución a X

- Se puede decir que el problema X no es más difícil que el problema Y
- Es importante que las transformaciones de los pasos 1 y 3 sean en tiempo polinomial, si no la reducción no es tan útil
- Las reducciones son transitivas
- Me permiten convertir un problema que no sé resolver en uno que sí

Complejidad



Créditos: Ron Jackson

Complejidad

- La máquina determinística de Turing es, a grandes rasgos, equiparable a cualquier computadora actual
- La máquina no determinística cuenta con infinitos procesadores. (Por ahora, es sólo teórica)
- Cada problema de optimización tiene asociado un problema de decisión, que se responde SI/NO
- Un certificador es un algoritmo que, dado un problema y una posible solución, nos dice si esa solución satisface el problema o no

- Con estas definiciones, podemos clasificar los problemas según su complejidad

Clase “P”

Un problema de decisión está en $\boldsymbol{P}$ *si una máquina* ***determinística*** *de Turing puede resolverlo en tiempo polinomial*

Clase “NP”

Un problema de decisión está en $\boldsymbol{NP}$ *si una máquina* ***no determinística*** *de Turing puede resolverlo en tiempo polinomial*

Un problema de decisión está en $\boldsymbol{NP}$ *si tiene un* ***algoritmo*** *que lo pueda certificar en una máquina* ***determinística*** *de Turing en tiempo polinomial*

Clase “NP-Completo”

Un problema de decisión X está en ***NP-Completo*** *sii todos los problemas en* ***NP*** *pueden reducirse a X*

Problemas NP-Hard

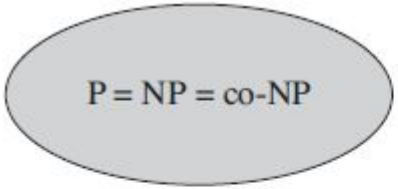
Si un problema de decisión X está en ***NP-Completo****, entonces el problema de optimización asociado es* ***NP-Hard***



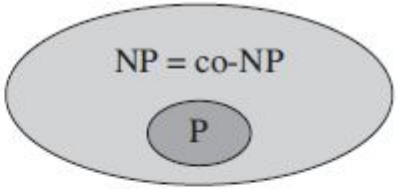
Clase “Co-NP”

Si un problema de decisión X está en $\boldsymbol{NP}$*, y su complementario* $\overline{X}$ *también está en* $\boldsymbol{NP}$*, entonces el problema X es un problema* $\boldsymbol{Co\text{-}NP}$ *(y su complementario también)*

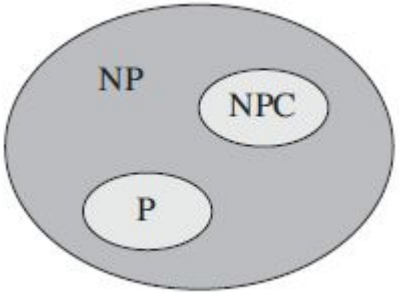
¿Cómo es el mundo?

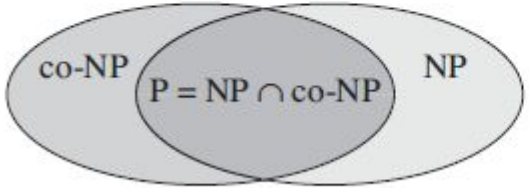


(a)

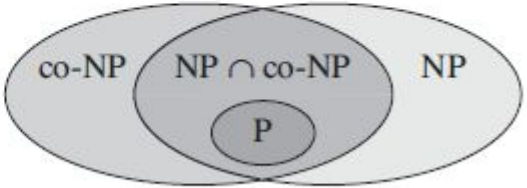


(b)



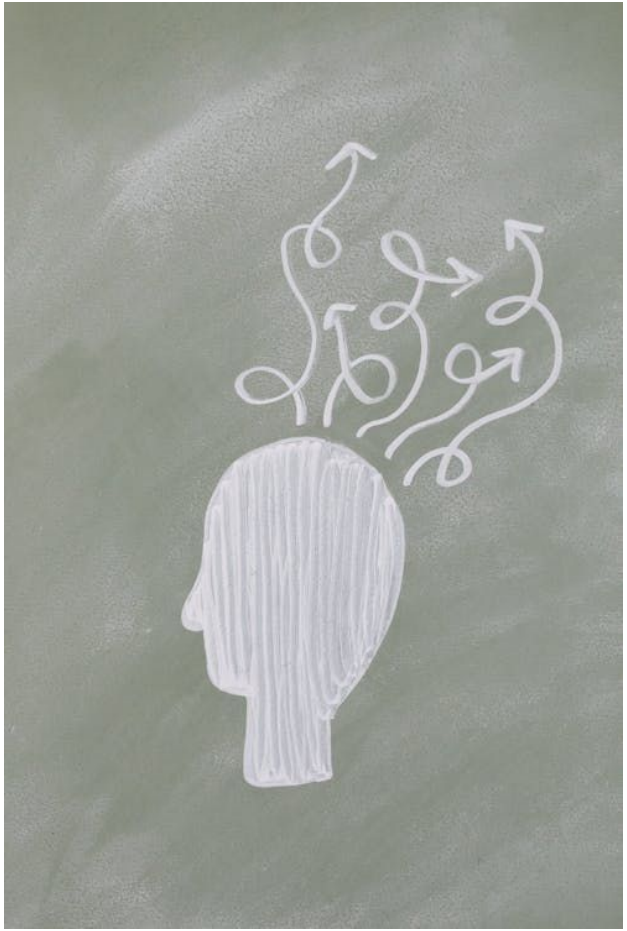


(c)



(d)

Problemas



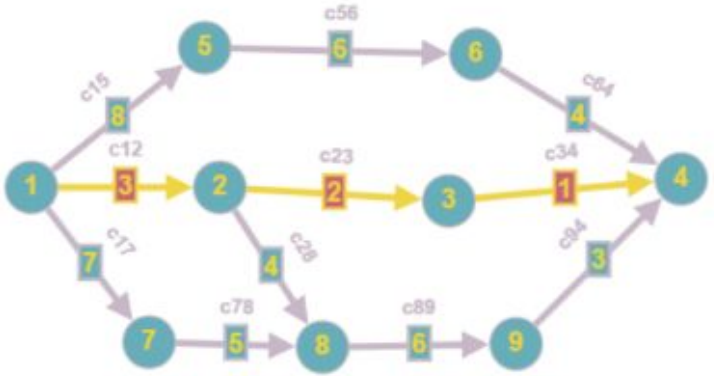
Créditos: Tara Winstead

Camino más corto

- Es un problema polinomial y los diferentes algoritmos encuentran el camino óptimo

Ejemplos:

- Algoritmo de Dijkstra (Greedy)
- Algoritmo de Prim (Greedy)
- Algoritmo de Kruskal (Greedy)
- Algoritmo de Bellman-Ford (PD)
- Algoritmo de Floyd-Warshall (PD)

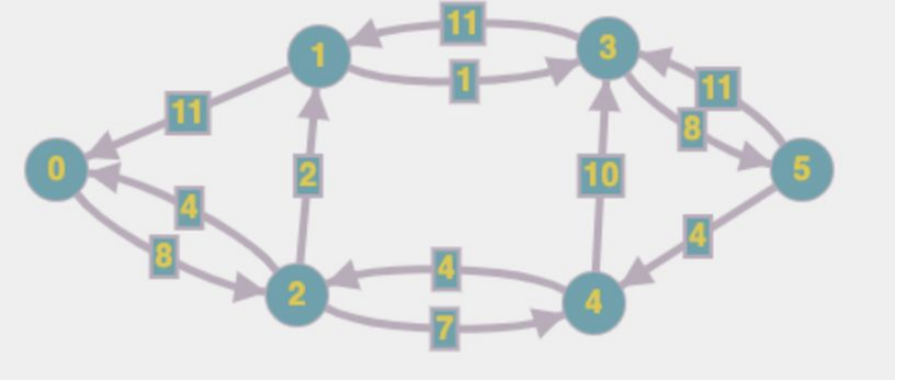


Redes de Flujo

- Es un problema polinomial y el algoritmo de Ford-Fulkerson encuentra el flujo máximo
- Flujo máximo = Corte mínimo

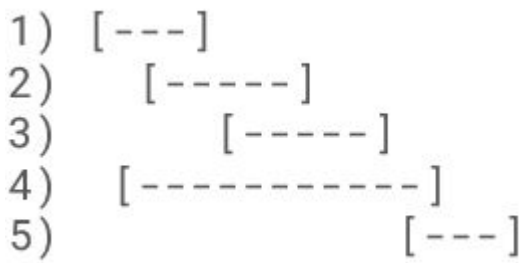
Ejemplos:

- Matching bipartito
- Caminos disjuntos
- Circulación con demanda
- Selección de proyectos



Scheduling

- Seleccionar tareas a cumplir en determinado período de tiempo
- En general, no se puede hacer todas
- En general, son polinomiales



Ejemplos:

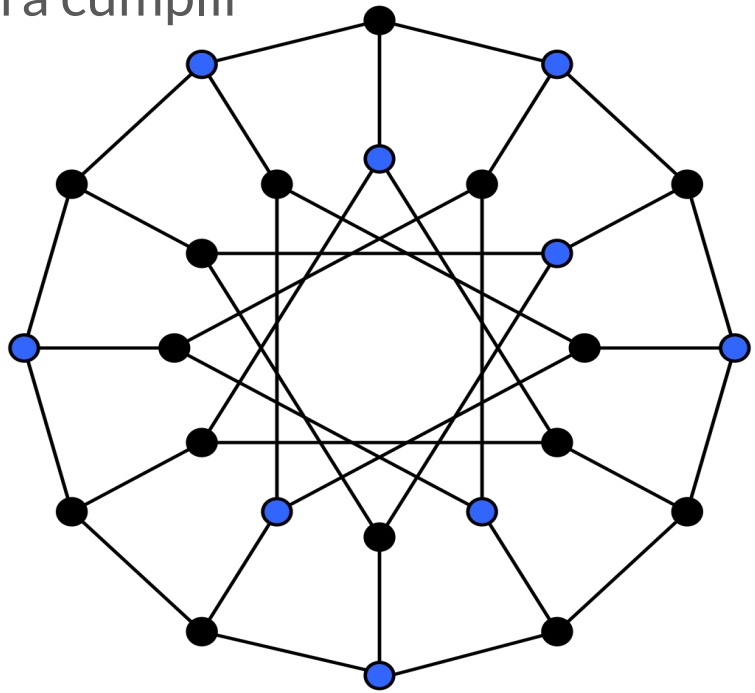
- Scheduling de Intervalos
- Scheduling con minimización de retrasos (hay que hacer todas)
- Coloreo de intervalos
- Scheduling de intervalos ponderados (Este no es polinomial)

Cobertura y Packing

- Seleccionar elementos de un grafo para cumplir con alguna restricción
- En general, se busca el mayor/menor
- No son polinomiales

Ejemplos:

- Independent set
- Vertex cover
- Set cover
- 3D Matching
- Coloreo de grafos



Créditos: Life of Riley

Selección de elementos

- Seleccionar elementos para cumplir con alguna restricción
- En general, se busca el mayor/menor
- No son polinomiales

Ejemplos:

- Mochila
- Subset Sum
- Corte de varilla (es polinomial)
- Cambio de monedas

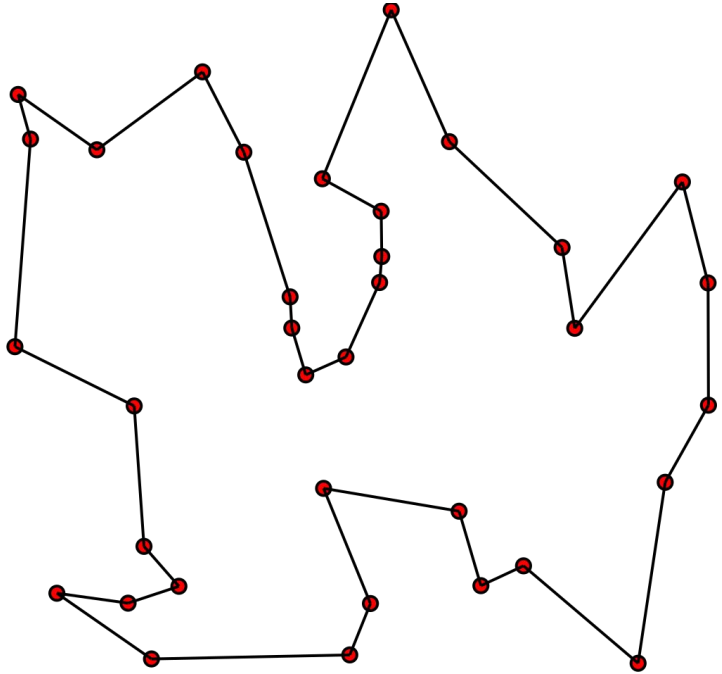


Secuencia

- Seleccionar elementos en un determinado orden
- No son polinomiales

Ejemplos:

- Camino Hamiltoniano
- Ciclo Hamiltoniano
- Viajante de Comercio

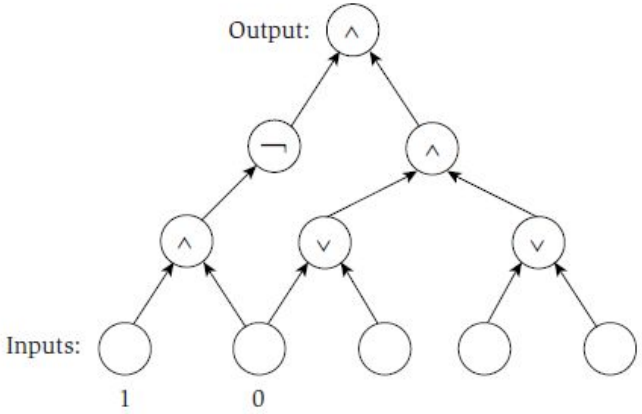


Satisfacción de restricciones

- Son los problemas más genéricos
  ¿Casi? todos los demás problemas pueden expresarse así
- No son polinomiales

Ejemplos:

- Satisfacibilidad de Circuitos
- SAT
- 3-SAT
- Programación lineal continua (sí es polinomial)
- Programación lineal entera



Resumen del Resumen

- Fuerza bruta / Backtracking
- División y conquista
- Greedy
- Programación dinámica
- Programación lineal
- Redes de Flujo
- Reducciones
- Algoritmos aproximados
- Algoritmos randomizados
- Autómatas y lenguajes

Resumen del Resumen

- Fuerza bruta / Backtracking
- División y conquista
- Greedy
- Programación dinámica
- Programación lineal
- Redes de Flujo
- Reducciones
- Algoritmos aproximados
- Algoritmos randomizados
- ~~Autómatas y lenguajes~~ → Autoevaluación por campus

Resumen del Resumen

- Fuerza bruta / Backtracking
- División y conquista
- Greedy
- Programación dinámica
- Programación lineal
- Redes de Flujo
- Reducciones
- ~~Algoritmos aproximados~~ → Recuperatorios
- ~~Algoritmos randomizados~~ → Recuperatorios
- ~~Autómatas y lenguajes~~

Ejercicios de Programación Lineal

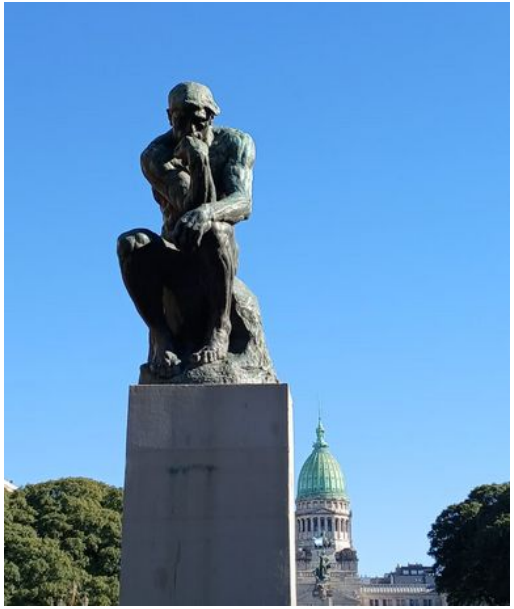


Problema 1

Plantear un modelo de Programación Lineal para resolver una Red de Flujo

Como estrategia general, pensar:

- ¿Cuáles van a ser las variables? ¿Qué medirán?
- ¿Hay constantes? ¿Qué indican?
- ¿Qué restricciones necesitamos plantear?
- ¿Cuál será la función objetivo a maximizar/minimizar?



"El Pensador" - A. Rodin

Problema 1 (cont.)

Variables:

- X_{ij}: cantidad de flujo que circula por la arista e_{ij}; entera

Constantes:

- C_{ij}: capacidad de la arista e_{ij}

Restricciones:

- Para cada nodo, todo lo que entra es igual a todo lo que sale
- El flujo de cada arista no debe exceder su capacidad

Función Objetivo:

- Maximizar el flujo saliente de la fuente s o el entrante en el sumidero t

Problema 1 (cont.)

Para cada vértice k, todo lo que entra es igual a todo lo que sale:

$$\sum_{i/\exists e_{ik} \in E} X_{ik} = \sum_{j/\exists e_{kj} \in E} X_{kj} \quad \forall k \in V$$

(Una restricción para cada vértice k)

Capacidad de cada arista:

$$X_{ij} \leq C_{ij} \quad \forall i, j / \exists e_{ij} \in E$$

(Una restricción para cada arista)

Función Objetivo:

$$max \sum_{j/e_{sj} \in E} X_{sj}$$

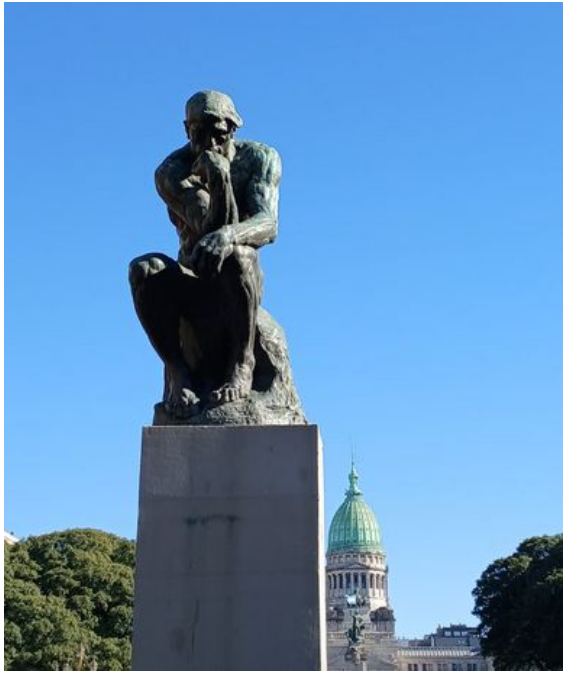
Problema 2

Plantear un modelo de Programación Lineal para determinar si el siguiente problema 3-SAT tiene solución:

- $C1: (x_1 \vee x_2 \vee x_3)$
- $C2: (\neg x_1 \vee x_2 \vee x_4)$
- $C3: (\neg x_4 \vee \neg x_3 \vee x_5)$

Problema 2 (cont.)

- ¿Cuáles van a ser las variables? ¿Qué medirán?
- ¿Hay constantes? ¿Qué indican?
- ¿Qué restricciones necesitamos plantear?
- ¿Cuál será la función objetivo a maximizar/minimizar?



“El Pensador” - A. Rodin

Problema 2 (cont.)

Variables:

- X_i: vale 1 si se asigna el valor TRUE a la variable x_i, y 0 si no; binaria

Constantes:

- No tenemos

Restricciones:

- Para cada cláusula, plantear un OR que deba ser siempre verdadero

Función Objetivo:

- Podemos maximizar o minimizar (no es relevante) la suma de las X_i

Problema 2 (cont.)

Cláusula C1: $(x_1 \vee x_2 \vee x_3)$

- $1 \leq X_1 + X_2 + X_3 \leq 3$

Cláusula C2: $(\neg x_1 \vee x_2 \vee x_4)$

- $1 \leq (1\text{-}X_1) + X_2 + X_4 \leq 3$

Cláusula C3: $(\neg x_4 \vee \neg x_3 \vee x_5)$

- $1 \leq (1\text{-}X_4) + (1\text{-}X_3) + X_5 \leq 3$

Función Objetivo:

Z(min) = $X_1 + X_2 + X_3 + X_4 + X_5$

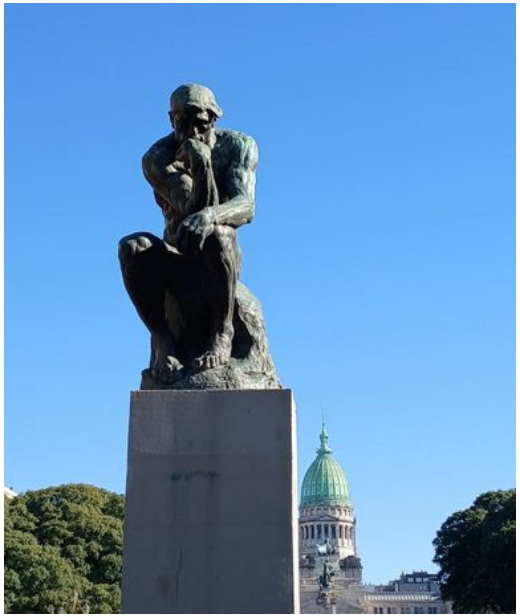
Ejercicios de Redes de Flujo



Problema 1

Se tiene una red de flujo ya resuelta y un flujo máximo ya calculado. Escribir un algoritmo de orden lineal que, incrementando la capacidad de una arista en 1, permita encontrar el nuevo flujo máximo. Justificar la complejidad.

1. Incrementar la capacidad de la arista en 1
2. Plantear un algoritmo que haga un BFS de s a t sobre el grafo residual modificado
3. El BFS es O(|V| + |E|)



"El Pensador" - A. Rodin

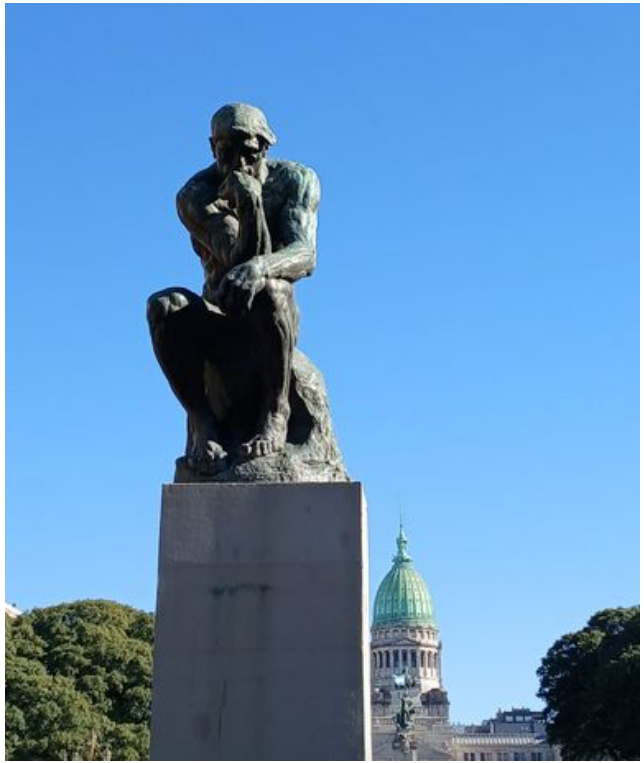
Problema 2

La WIDS (Women in Data Science) quiere formar un panel de evaluación de trabajos. Sus miembros serán docentes. Cada docente tiene una única especialidad y dicta clases en una o más universidades. Se desea formar el panel de manera tal que cada universidad esté representada y que a lo sumo haya N/2 representantes de cada especialidad (siendo N la cantidad total de universidades en las que dan clase las docentes).

Implementar un algoritmo basado en Redes de Flujo que nos de la lista de miembros del panel.

Problema 2 (cont.)

¿Ideas?

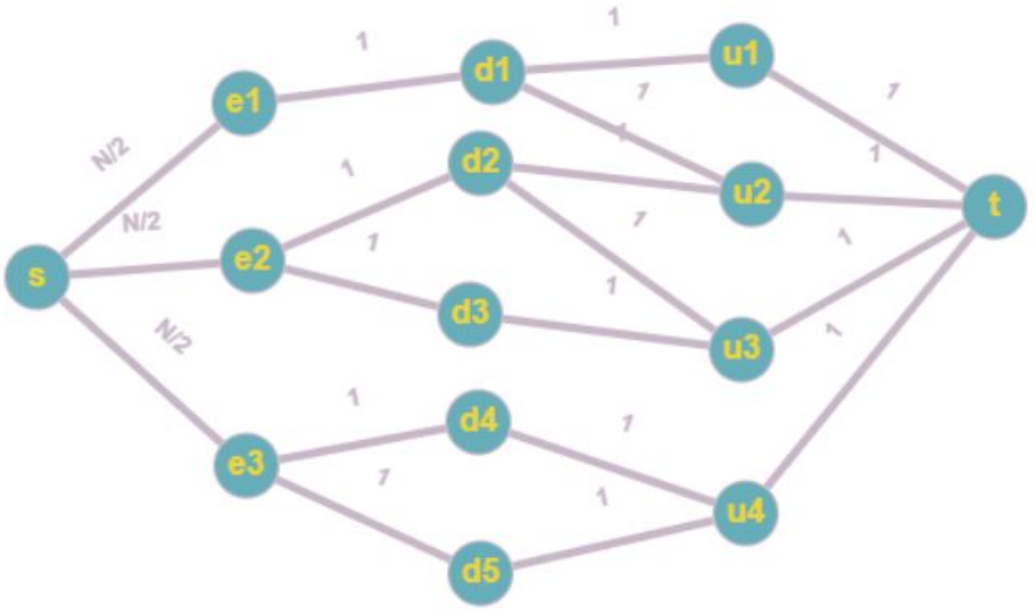


“El Pensador” - A. Rodin

Problema 2 (cont.)

Construir una Red de Flujo:

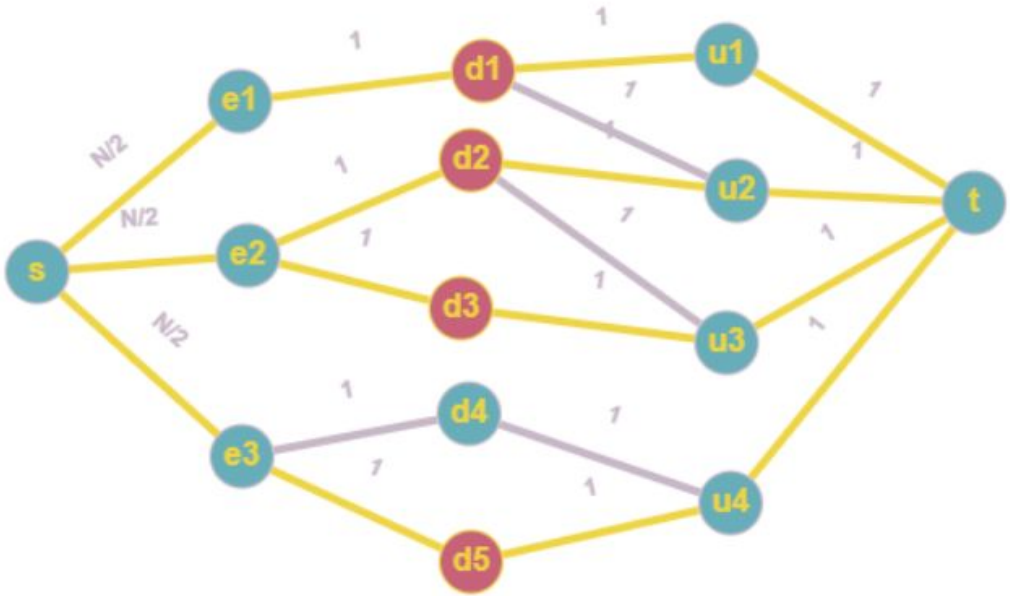
- e_i: especialidad
- d_i: docente
- u_i: Universidad



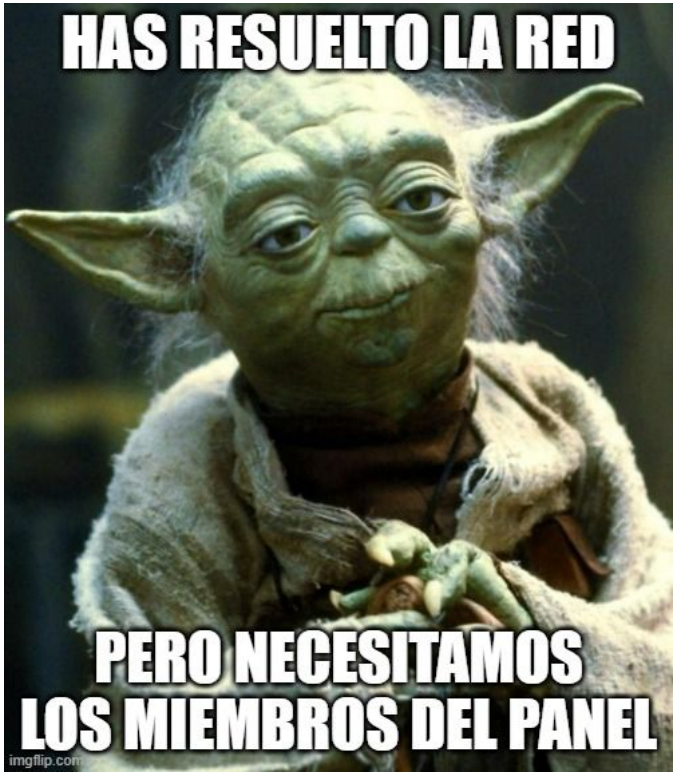
Problema 2 (cont.)

Ok, tenemos la Red de Flujo

Busquemos el flujo máximo con Ford Fulkerson

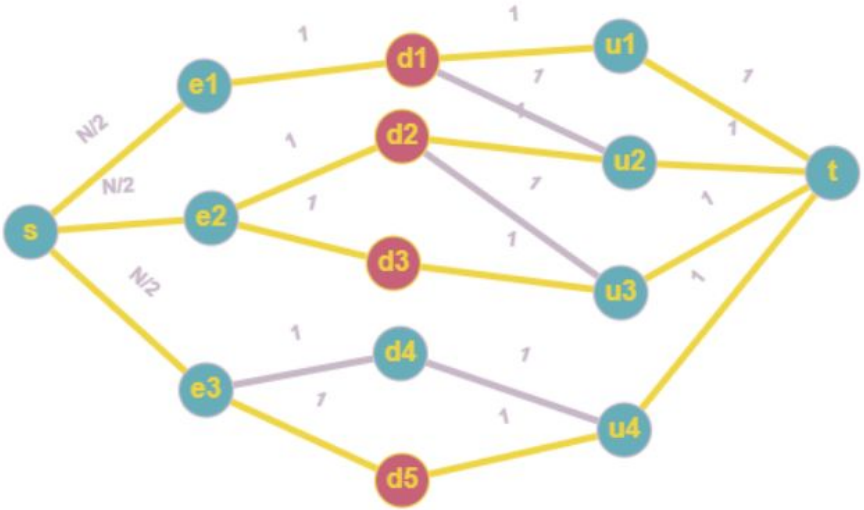


Problema 2 (cont.)



Problema 2 (cont.)

1. Construir el grafo de la Red de Flujo
2. Resolver con FF
3. Para cada docente:
   a. Si el flujo saliente hacia el nodo de la universidad es 1
      i. Agregarlo a la lista del panel



Ejercicios de Programación Dinámica



Problema 1

Se tiene una soga de longitud n≥2 y se desea cortarla en dos o más partes, tal que el producto de las partes sea el máximo. Implementar un algoritmo de Programación Dinámica que devuelva el valor del producto máximo para un n entero dado.

Ejemplos:

- n=2: devuelve 1 (1*1)
- n=3: devuelve 2 (2*1)
- n=4: devuelve 4 (2*2)
- n=5: devuelve 6 (2*3)
- n=10: devuelve 36 (3*3*4)

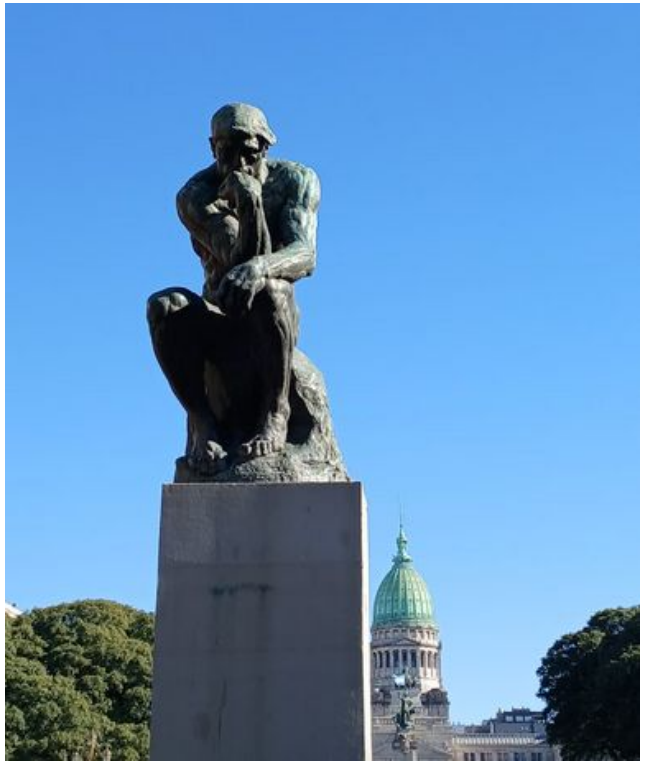
Problema 1 (cont.)

Estrategia:

1. Pensar los casos base
2. Plantear la ecuación de recurrencia
3. Comprobar que se cumplan los dos requisitos de PD:
    a. Subestructura Óptima
    b. Subproblemas Superpuestos
4. Memoization: ¿qué me conviene almacenar para uso posterior?
5. Plantear el algoritmo

Problema 1 (cont.)

¿Ideas?



“El Pensador” - A. Rodin

Problema 1 (cont.)

Casos Base:

- n=2: devuelve 1 (1*1)
- n=3: devuelve 2 (2*1)

Ecuación de Recurrencia (n > 3):

$$OPT(n) = \max_{i/i \in [2,n-1]} \left\{ max \begin{cases} i * (n-i), \\ i * OPT(n-i), \\ OPT(i) * (n-i), \\ OPT(i) * OPT(n-i) \end{cases} \right\}$$

¿Hay Subestructura Óptima?

¿Hay Subproblemas Superpuestos?

¿Dónde hago Memoization?

¡Muchas gracias!

